Modern Transformer Upgrades

RoPE, RMSNorm, SwiGLU, GQA, KV-Cache & Flash Attention

1

Where We Are

- ✅ Part 1: The **original Transformer** (Vaswani et al., 2017) — attention, FFN, residuals, layer norm, sinusoidal PE
- ✅ Part 2: **NLP fundamentals** — tokenization, embeddings, output pipeline, loss functions, metrics
- ❓ Today: **Modern transformer upgrades** — the 6 key changes that turn the 2017 design into a production-grade LLM backbone
- ➡️ SOON Next: **GPT-OSS from scratch** — data pipeline, tokenizer training, model implementation, pre-training loop
- Today's goal: understand **every architectural upgrade** so you can read any modern LLM's code and know exactly what each piece does

2

Overview — From 2017 to 2026

3

Why Upgrade the Original Transformer?

- The 2017 Transformer **works** — but it was designed for machine translation at moderate scale
- Modern LLMs train on **trillions of tokens** with **billions+ of parameters** and (in many systems) **very long contexts**
- At this scale, small inefficiencies in **training stability**, **memory**, and **compute** become critical
- Every upgrade we cover today solves a **specific, concrete problem** that emerges at scale:
- • Training instability in deep networks → **Pre-Norm**
- • Unnecessary computation in normalization → **RMSNorm**
- • Suboptimal FFN expressiveness → **SwiGLU**
- • Poor length generalization → **RoPE**
- • KV-cache memory explosion at inference → **GQA** (and, in newer stacks, MLA-style KV compression + sparse attention)
- • Quadratic memory in attention → **Flash Attention** (and related fused SDPA kernels)
- These are **drop-in replacements** — the overall architecture and pipeline stay exactly the same

The Upgrade Map

- **Original (2017) → Modern (2024–2026)** — component by component:
- • **Post-LayerNorm → Pre-RMSNorm** — normalization placement and type
- • **Sinusoidal / Learned positions → RoPE** — position encoding method
- • **Multi-Head Attention (MHA) → GQA / KV-sharing variants** — smaller KV-cache
- • **ReLU FFN → SwiGLU FFN** — feed-forward activation and gating
- • **No KV reuse → KV-Cache** — inference-time optimization
- • **Standard attention → Flash Attention / SDPA** — memory-efficient attention kernel
- • **Biases everywhere → No biases** (often) — minor simplification, slightly fewer parameters
- • **Dropout → No dropout** (often, in pre-training) — less needed at huge data scale
- Examples with public configs/tech reports: **Qwen3.5-397B-A17B, GLM-5, Kimi K2.5, MiniMax M2.5, DeepSeek V3.2** — all implement this modern core (then diverge with MoE + hybrid/sparse/MLA attention).

5

A Concrete Before/After

- Imagine training a 7B-parameter model on 2T tokens with 128K context. What changes?
- **Training stability**: Post-norm diverges at layer 60+; Pre-RMSNorm trains smoothly to 128 layers
- **Normalization cost**: Switching LayerNorm → RMSNorm saves ~15% norm time → **~2 days** on a 2-week run
- **FFN quality**: SwiGLU reaches the same validation loss as ReLU with **~10% fewer tokens** — weeks of GPU time
- **Position generalization**: Learned PE caps at training length; RoPE extends to 4× with simple frequency scaling
- **Inference memory**: MHA KV-cache at 128K = 64 GB; GQA (8 KV heads) = 8 GB — fits on one GPU
- **Attention memory**: Standard attention at 128K = 16B entries per head; Flash Attention = O(T), never materialized
- None of these change **what** the model computes at a high level — they change **how efficiently** it computes it

6

Part 1: Pre-Norm — Stabilizing Deep Networks

7

Post-Norm vs Pre-Norm

- **Post-Norm** (Original 2017): Normalize **after** the residual addition
- $x_{\text{out}} = \text{Norm}(x + \text{Sublayer}(x))$
- **Pre-Norm** (Modern): Normalize **before** the sublayer; residual bypasses the norm entirely
- $x_{\text{out}} = x + \text{Sublayer}(\text{Norm}(x))$
- **Why Pre-Norm wins at scale:**
- In Post-Norm, gradients must traverse the Norm at every layer — $\frac{\partial \mathcal{L}}{\partial x_l}$ passes through $N - l$ normalizations
- In Pre-Norm, the residual stream is **unobstructed** — a clean gradient highway from last layer to first
- Result: **Pre-Norm trains much more stably**, especially for deep models (32–128 layers)
- Post-Norm often diverges without careful learning rate warmup; Pre-Norm rarely needs it
- Also requires a **final RMSNorm** after the last transformer layer, before the LM head
- Popularized by Xiong et al. (2020); now widely used in modern decoder-only LLM backbones

8

Pre-Norm vs Post-Norm in Code

PYTHON

```
import torch.nn as nn

# --- POST-NORM (Original 2017 Transformer) ---
class PostNormBlock(nn.Module):
    def __init__(self, embed_dim, n_heads):
        super().__init__()
        self.attention = MultiHeadAttention(embed_dim, n_heads)
        self.ffn = FFN(embed_dim)
        self.norm1 = nn.LayerNorm(embed_dim)
        self.norm2 = nn.LayerNorm(embed_dim)

    def forward(self, x):
        x = self.norm1(x + self.attention(x))  # Norm AFTER residual
        x = self.norm2(x + self.ffn(x))        # Gradient must flow thro
        return x                                # at EVERY layer → unsta

# --- PRE-NORM (Modern — common in 2025–2026 SOTA codebases) ---
class PreNormBlock(nn.Module):
    def __init__(self, embed_dim, n_heads):
        super().__init__()
        self.attention = MultiHeadAttention(embed_dim, n_heads)
        self.ffn = FFN(embed_dim)
        self.norm1 = RMSNorm(embed_dim)  # RMSNorm, not LayerNorm!
        self.norm2 = RMSNorm(embed_dim)

    def forward(self, x):
        x = x + self.attention(self.norm1(x))  # Norm BEFORE sublayer
        x = x + self.ffn(self.norm2(x))        # Clean residual highway
        return x                                # Stable even at 100+ la
```

Part 2: RMSNorm — Faster Normalization

10

From LayerNorm to RMSNorm

- **LayerNorm** (Ba et al., 2016) — used in the original Transformer:
- $\mathrm{LayerNorm}(x) = \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} \cdot \gamma + \beta$
- Computes **mean** μ and **variance** σ^2 across the feature dimension, then re-scales and re-centers
- Two learned parameters: γ (scale) and β (bias), each of dimension d
- **RMSNorm** (Zhang & Sennrich, 2019) — used in many modern LLMs:
- $\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\mathrm{mean}(x^2)+\epsilon}} \cdot \gamma$
- Only computes the **root mean square** — no mean subtraction, no learned bias β
- **Why switch?** The re-centering (mean subtraction) in LayerNorm was found to be **unnecessary** — removing it often doesn't hurt quality
- **Fewer operations**: No mean computation, no mean subtraction, no bias addition
- RMSNorm is **~10–15% faster** due to fewer operations, fewer parameters (no β)
- At billions of parameters and trillions of tokens, this speedup adds up to **days of saved training time**
- Concrete examples in public configs: **Qwen3.5** (`rms_norm_eps=1e-6`), **GLM-5** (`rms_norm_eps=1e-5`), **Kimi K2.5** (`rms_norm_eps=1e-5`), **DeepSeek V3.2** (`rms_norm_eps=1e-6`), **MiniMax M2.5** (`rms_norm_eps=1e-6`).

11

RMSNorm Implementation

PYTHON

```
import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    """Root Mean Square Layer Normalization (Zhang & Sennrich, 2019).
    Drop-in replacement for nn.LayerNorm — fewer ops, same quality."""
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))  # Learnable scale γ
        # Note: NO bias parameter β (unlike LayerNorm)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x shape: (batch, seq_len, dim)
        rms = torch.sqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        x_normed = x / rms                      # Normalize by RMS
        return x_normed * self.weight             # Scale by learned γ

# Compare parameter counts:
dim = 4096
layer_norm = nn.LayerNorm(dim)     # 2 × 4096 = 8,192 params (γ + β)
rms_norm   = RMSNorm(dim)          # 1 × 4096 = 4,096 params (γ only)
print(f"LayerNorm params: {sum(p.numel() for p in layer_norm.parameters(
print(f"RMSNorm params:   {sum(p.numel() for p in rms_norm.parameters())

# Per model (32 layers × 2 norms each = 64 norm layers):
# LayerNorm: 64 × 8,192 = 524,288 params
# RMSNorm:   64 × 4,096 = 262,144 params — half the norm parameters
```

Part 3: SwiGLU — The Modern Feed-Forward Network

13

The FFN Evolution: ReLU → GELU → Swish → SwiGLU

- Recall the original FFN: $\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$ — two linear layers with ReLU
- **ReLU** ($\max(0, x)$): Simple but has a "dead neuron" problem — once a neuron outputs 0, its gradient is 0 forever
- **GELU** ($x \cdot \Phi(x)$): Smooth approximation of ReLU, used in GPT-2 and BERT — no dead neurons
- **Swish / SiLU** ($x \cdot \sigma(x)$): Similar to GELU but simpler — smooth, non-monotonic, slightly better empirically
- **But the real upgrade isn't just the activation function — it's adding a gating mechanism:**
- **Gated Linear Units (GLU)** (Dauphin et al., 2017): Split the hidden state in two — one half **gates** the other
- **SwiGLU** = Swish activation + Gated Linear Unit — the common choice in many modern decoder stacks (often implemented as gate_proj/up_proj/down_proj with SiLU)
- Shazeer (2020): "GLU Variants Improve Transformer" — SwiGLU **consistently outperforms** ReLU and GELU across model sizes

🚀 **Interactive Demo:** activation_functions_demo.html

14

SwiGLU: Gated Activation for Better FFNs

- **Standard FFN** (2 weight matrices):
- $\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1) \cdot W_2$
- **SwiGLU FFN** (3 weight matrices):
- $\mathrm{SwiGLU}(x) = (\mathrm{Swish}(xW_1) \odot xW_3) \cdot W_2$
- where $\mathrm{Swish}(x) = x \cdot \sigma(x)$ and $\odot$ is element-wise multiplication
- W_1 (gate projection): produces the **gate signal** — how much of each feature to let through
- W_3 (up projection): produces the **value signal** — what information to potentially pass forward
- W_2 (down projection): projects back down to model dimension
- **3 matrices instead of 2** → to keep total parameter count similar, hidden dim is adjusted: $h = \frac{8}{3}d$ instead of $4d$
- Example: $d = 4096$ → original hidden = 16,384; SwiGLU hidden ≈ 10,923 (rounded to 11,008 for hardware alignment)
- Intuition: the gate learns **when** to activate each feature, not just **how much** — a richer, more expressive transformation
- **No bias terms** in many modern implementations — biases are often removed throughout

15

SwiGLU FFN Implementation

PYTHON

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SwiGLUFFN(nn.Module):
    """SwiGLU Feed-Forward Network (Shazeer 2020).
    Replaces the standard ReLU/GELU FFN in modern transformers."""
    def __init__(self, embed_dim: int, hidden_dim: int = None):
        super().__init__()
        if hidden_dim is None:
            hidden_dim = int(8 * embed_dim / 3)                # Compens
            hidden_dim = 256 * ((hidden_dim + 255) // 256)     # Round t
        self.w1 = nn.Linear(embed_dim, hidden_dim, bias=False)  # Gate p
        self.w3 = nn.Linear(embed_dim, hidden_dim, bias=False)  # Up pro
        self.w2 = nn.Linear(hidden_dim, embed_dim, bias=False)  # Down p

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # SwiGLU(x) = (Swish(xW₁) ⊙ xW₃) · W₂
        return self.w2(F.silu(self.w1(x)) * self.w3(x))  # F.silu = Swis

# Parameter comparison at embed_dim = 4096:
class OriginalFFN(nn.Module):
    def __init__(self, d, h=None):
        super().__init__()
        h = h or 4 * d  # h = 16384
        self.w1 = nn.Linear(d, h, bias=False)
        self.act = nn.ReLU()
        self.w2 = nn.Linear(h, d, bias=False)
    def forward(self, x): return self.w2(self.act(self.w1(x)))

original = OriginalFFN(4096)       # 2 matrices: 4096×16384 × 2 = 134,21
swiglu   = SwiGLUFFN(4096)         # 3 matrices: 4096×11008 × 2 + 11008×
print(f"Original: {sum(p.numel() for p in original.parameters()):>12,} p
print(f"SwiGLU:   {sum(p.numel() for p in swiglu.parameters()):>12,} par
# Nearly identical parameter count — but SwiGLU performs significantly b
```

16

Part 4: RoPE — Rotary Position Embeddings

17

Why Absolute Position Embeddings Fall Short

- **Learned positional embeddings** (GPT-2 style): a lookup table of shape $(T_{\max} \times d)$
- Position 0 gets one learned vector, position 1 gets another, ..., position $T_{\max} - 1$ gets the last one
- **Problem 1 — Fixed maximum length**: Cannot process sequences longer than $T_{\max}$ seen during training
- **Problem 2 — No relative distance**: The model must *learn* that positions 5 and 7 are close — this isn't built in
- The relationship between "cat" and "sat" should depend on their **distance** (2 apart), not absolute positions
- **Problem 3 — No parameter sharing**: Position 500 has no structural relationship to position 501 — each is independent
- **What we want**: Attention scores $q_m \cdot k_n$ that naturally depend on the **relative distance** $(m - n)$
- **RoPE** achieves exactly this — using rotation to encode position, with no learned position parameters

18

RoPE: The Core Idea — Rotation Encodes Position

- **Key insight**: If we **rotate** the query at position m by angle $m\theta$, and the key at position n by angle $n\theta$...
- ...then their dot product depends only on $(m-n)\theta$ — the **relative distance**!
- **In 2D**: Rotating vector $[x_1, x_2]$ by angle α:
- $x_1' = x_1 \cos\alpha - x_2 \sin\alpha, \quad x_2' = x_1 \sin\alpha + x_2 \cos\alpha$
- If q is rotated by $m\theta$ and k by $n\theta$: $q_m \cdot k_n$ depends on $(m-n)\theta$ ✓
- **High dimensions**: Split the d-dimensional head vector into $d/2$ pairs — each pair is a 2D plane
- Each pair gets a **different rotation frequency**: $\theta_i = 10000^{-2i/d}$ for $i = 0, 1, \ldots, d/2 - 1$
- Low-frequency pairs → capture **coarse, long-range** position; High-frequency pairs → capture **fine, local** position
- Applied to **Q and K only** (not V) — position affects *where* to attend, not *what* to aggregate

🚀 **Interactive Demo:** rope_demo.html

19

RoPE: The Mathematics

- For a head of dimension d, at position m, RoPE transforms a vector x as follows:
- Split x into $d/2$ consecutive pairs: $(x_0, x_1), (x_2, x_3), \ldots, (x_{d-2}, x_{d-1})$
- For pair i, apply 2D rotation by angle $m \cdot \theta_i$:
- $\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix}$
- Frequency schedule: $\theta_i = 10000^{-2i/d}$ — logarithmically spaced from high to low frequency
- **Elegant implementation**: Treat each pair as a **complex number** $(x_{2i} + j \cdot x_{2i+1})$, then multiply by $e^{jm\theta_i}$
- Complex multiplication **IS** 2D rotation — this makes the code very clean
- **No learned parameters** — the rotation angles are determined entirely by position and the fixed frequency schedule

20

RoPE: Key Properties and Benefits

- ✅ **Relative position encoding**: $q_m^T k_n$ depends on $(m - n)$ — relative distance is built into the dot product
- ✅ **No learned parameters**: Zero additional parameters for position — the frequency schedule is fixed
- ✅ **Natural distance decay**: Attention naturally decays with distance — a soft locality bias emerges
- ✅ **Length extrapolation**: Can handle sequences longer than training length with frequency scaling techniques
- **Context length extension**: Many 2025–2026 stacks use larger RoPE base θ plus scaling tricks
- Concrete examples from public configs:
- • **Qwen3.5-397B-A17B**: `max_position_embeddings=262144`, `rope_theta=10000000`, `partial_rotary_factor=0.25`
- • **Kimi K2.5**: `max_position_embeddings=262144`, `rope_theta=50000`, `rope_scaling.type="yarn"`, `rope_scaling.factor=64`
- • **GLM-5**: `max_position_embeddings=202752`, `rope_parameters.rope_theta=1000000`
- • **MiniMax M2.5**: `max_position_embeddings=196608`, `rope_theta=5000000`, `rotary_dim=64`
- • **DeepSeek V3.2**: `max_position_embeddings=163840`, `rope_theta=10000`, `rope_scaling.type="yarn"`, `rope_scaling.factor=40`
- Advanced techniques: **YaRN**, **NTK-aware scaling** — modify frequency schedule to extend context further
- Introduced by Su et al. (2021): "RoFormer: Enhanced Transformer with Rotary Position Embedding"
- Note: With RoPE, there is **no positional embedding table** — no `nn.Embedding(max_seq_len, embed_dim)` needed!

RoPE Implementation

PYTHON

```
import torch

def precompute_freqs_cis(head_dim: int, max_seq_len: int, theta: float =
    """Precompute rotation frequencies as complex exponentials."""
    freqs = 1.0 / (theta ** (torch.arange(0, head_dim, 2).float() / heac
    positions = torch.arange(max_seq_len)
    angles = torch.outer(positions, freqs)           # (max_seq_len, hea
    return torch.polar(torch.ones_like(angles), angles)  # e^(jθ) = cos

def apply_rope(q, k, freqs_cis):
    """Apply rotary embeddings to query and key tensors.
    q, k: (batch, seq_len, n_heads, head_dim)
    freqs_cis: (seq_len, head_dim/2) — precomputed complex exponentials
    """
    # Reshape to pairs → view as complex numbers
    q_complex = torch.view_as_complex(q.float().reshape(*q.shape[:-1], -
    k_complex = torch.view_as_complex(k.float().reshape(*k.shape[:-1], -

    # Broadcast freqs_cis: (seq_len, head_dim/2) → (1, seq_len, 1, head_
    freqs = freqs_cis.unsqueeze(0).unsqueeze(2)

    # Complex multiplication = 2D rotation!
    q_rotated = torch.view_as_real(q_complex * freqs).flatten(-2)
    k_rotated = torch.view_as_real(k_complex * freqs).flatten(-2)
    return q_rotated.type_as(q), k_rotated.type_as(k)

# Precompute once at model init:
freqs_cis = precompute_freqs_cis(head_dim=128, max_seq_len=8192)
# Then in each attention layer:
# q, k = apply_rope(q, k, freqs_cis[:seq_len])
# Followed by standard attention: softmax(QK^T / √d) V
```

Part 5: GQA — Grouped Query Attention

23

The KV Memory Problem at Scale

- Recall **Multi-Head Attention (MHA)**: each head has its own Q, K, and V projections
- During autoregressive generation, we **cache** K and V from all previous positions (KV-cache — covered next)
- KV-cache size per layer = $2 \times T \times n_{\text{heads}} \times d_{\text{head}} \times \text{bytes}$
- For a 70B-class model: 64 heads, 128 dim/head, 80 layers, float16:
- KV-cache at 4K context = $2 \times 4096 \times 64 \times 128 \times 2 \times 80$ ≈ **10.7 GB** just for the cache!
- At 128K context → **~340 GB** — larger than the model weights themselves!
- The KV-cache is often the **inference bottleneck** — not compute, but memory capacity and bandwidth
- **Key question**: Do all heads really need their own independent K and V? Or can we share?

24

From MHA to MQA to GQA

- **Multi-Head Attention (MHA)**: n_h Q heads, n_h K heads, n_h V heads — full independence
- Every head has its own KV projection → maximum expressiveness, maximum memory cost
- **Multi-Query Attention (MQA)** (Shazeer 2019): n_h Q heads, **1** K head, **1** V head
- All query heads share a **single** K and V → dramatic memory reduction ($n_h\times$ less KV-cache)
- But quality degrades noticeably — too much information sharing
- **Grouped Query Attention (GQA)** (Ainslie et al. 2023): n_h Q heads, n_{kv} K heads, n_{kv} V heads
- Groups of Q heads share KV heads — the **sweet spot** between MHA and MQA
- e.g., 32 Q heads, 8 KV heads → groups of 4 Q heads share each KV head
- KV-cache reduction = n_h/n_{kv} = 4× — with **negligible quality loss**
- GQA with $n_{kv} = 1$ = MQA; GQA with $n_{kv} = n_h$ = standard MHA — it's a spectrum

25

GQA: Visual Intuition

- **MHA** (32 heads): $Q_1K_1V_1$, $Q_2K_2V_2$, $Q_3K_3V_3$, ... $Q_{32}K_{32}V_{32}$ — **32 KV pairs**
- **GQA** (32 Q heads, 8 KV groups): $\{Q_1Q_2Q_3Q_4\}K_1V_1$, $\{Q_5Q_6Q_7Q_8\}K_2V_2$, ... $\{Q_{29}Q_{30}Q_{31}Q_{32}\}K_8V_8$ — **8 KV pairs**
- **MQA** (32 Q heads, 1 KV group): $\{Q_1Q_2...Q_{32}\}K_1V_1$ — **1 KV pair**
- Queries remain **unique per head** — the model retains diverse "questions"
- Keys and values are **shared within groups** — the "memory" is compressed
- **Why this works**: Experiments show K and V are more redundant across heads than Q
- Different heads often attend to similar positions — sharing KV captures this without waste
- The Q projection is full-size; only the K and V projections are smaller

🚀 **Interactive Demo:** gqa_demo.html

26

GQA in Practice

- **How it works**: Repeat (broadcast) each KV head to match its group of Q heads before computing attention
- **Clean GQA examples (explicit in public configs):**
- • **Qwen3.5-397B-A17B (text full-attention blocks)**: `num_attention_heads=32`, `num_key_value_heads=2` → 16:1 KV sharing in those blocks
- • **MiniMax M2.5**: `num_attention_heads=48`, `num_key_value_heads=8` → 6:1 KV sharing
- **Beyond classic GQA:**
- • **DeepSeek V3.2**: introduces **DeepSeek Sparse Attention (DSA)** with a **Lightning Indexer**, and the tech report explicitly states DSA is instantiated under **MLA**; the paper also illustrates MHA/MQA modes of MLA
- • **Kimi K2.5**: model card lists attention mechanism as **MLA** (a KV-compression family of ideas)
- • **GLM-5**: integrates **DSA** (per model card) and uses RoPE + RMSNorm; it also exposes indexer-related config fields similar in spirit to sparse selection
- GQA reduces **KV parameters** + **KV-cache memory** + often improves **decode throughput**

Grouped Query Attention Implementation

PYTHON

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class GroupedQueryAttention(nn.Module):
    """GQA: Multiple Q heads share fewer KV heads (Ainslie et al. 2023).
    def __init__(self, embed_dim: int, n_heads: int, n_kv_heads: int):
        super().__init__()
        assert n_heads % n_kv_heads == 0, "n_heads must be divisible by
        self.n_heads = n_heads
        self.n_kv_heads = n_kv_heads
        self.n_rep = n_heads // n_kv_heads   # Q heads per KV group
        self.head_dim = embed_dim // n_heads

        self.wq = nn.Linear(embed_dim, n_heads * self.head_dim, bias=Fal
        self.wk = nn.Linear(embed_dim, n_kv_heads * self.head_dim, bias=
        self.wv = nn.Linear(embed_dim, n_kv_heads * self.head_dim, bias=
        self.wo = nn.Linear(n_heads * self.head_dim, embed_dim, bias=Fal

    def forward(self, x, freqs_cis):
        B, T, _ = x.shape
        q = self.wq(x).view(B, T, self.n_heads, self.head_dim)
        k = self.wk(x).view(B, T, self.n_kv_heads, self.head_dim)
        v = self.wv(x).view(B, T, self.n_kv_heads, self.head_dim)

        q, k = apply_rope(q, k, freqs_cis)   # RoPE on Q and K only

        # Repeat KV heads to match Q head count
        k = k.repeat_interleave(self.n_rep, dim=2)  # (B, T, n_heads, he
        v = v.repeat_interleave(self.n_rep, dim=2)

        # Standard scaled dot-product attention (uses fast SDPA kernels
        q, k, v = [t.transpose(1, 2) for t in (q, k, v)]  # (B, n_heads,
        out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        out = out.transpose(1, 2).contiguous().view(B, T, -1)
        return self.wo(out)

# Parameter savings (embed_dim=4096, 32 Q heads, 8 KV heads, head_dim=12
# MHA K proj: 4096 × 4096 = 16,777,216   GQA K proj: 4096 × 1024 = 4,194
```

GQA: Memory Savings Quantified

- Consider: 32 layers, 32 query heads, head_dim=128, sequence length=128K, float16 (2 bytes/element)
- **MHA** (32 KV heads): KV-cache = $32 \times 2 \times 32 \times 128\text{K} \times 128 \times 2$ bytes = **32 GB**
- **GQA** (8 KV heads): KV-cache = $32 \times 2 \times 8 \times 128\text{K} \times 128 \times 2$ bytes = **8 GB — 4× reduction!**
- **MQA** (1 KV head): KV-cache = $32 \times 2 \times 1 \times 128\text{K} \times 128 \times 2$ bytes = **1 GB** — but quality loss
- At long context lengths, the KV-cache dominates GPU memory — GQA makes long-context inference feasible
- **Quality impact**: GQA-8 matches MHA-32 quality on most benchmarks (Ainslie et al., 2023)
- This is why GQA is a standard production choice for long-context decoding — high quality at a fraction of the memory

29

Three KV-Memory Strategies (2025–2026): Sharing vs Compression vs Selection

- Modern long-context stacks reduce KV cost in **three distinct ways**. They can be combined.
-
- **1) KV-sharing (GQA / MQA)** — *store fewer KV heads*
- • Mechanism: many Q heads share the same K/V heads (GQA: n_kv < n_q; MQA is the extreme n_kv = 1)
- • KV-cache size scales with **n_kv** → savings ≈ **n_q / n_kv**
- • Tradeoff: too much sharing (MQA) can hurt quality; GQA is the usual sweet spot
- • Examples (public configs): Qwen3.5 full-attention blocks, MiniMax M2.5
-
- **2) KV-compression (MLA / low-rank latent KV)** — *store a smaller latent, reconstruct KV*
- • Mechanism: compress past information into a **latent** (rank r ≪ d) instead of caching full K/V per head
- • KV-cache size scales with the **latent size** (and sometimes a small per-head adapter), not full per-head K/V
- • Tradeoff: more complexity + extra projection work, but can enable much longer context at similar memory
- • Examples (official docs/tech reports): DeepSeek V3.2 uses MLA; Kimi K2.5 lists MLA
-
- **3) KV-selection (sparse attention like DSA top-k + indexer)** — *attend to a subset of past tokens*
- • Mechanism: pick **top-k** relevant past positions (using an indexer/scoring), then run attention on only those
- • Compute + bandwidth scale with **k** instead of T (sequence length)
- • Tradeoff: not full attention everywhere; quality depends on the selector/indexer doing the right retrieval

- • Example (official tech report): DeepSeek V3.2 introduces DSA with a Lightning Indexer + Top-k selector
-
- **Rule of thumb:**
- • GQA/MQA = "same memory, fewer copies"
- • MLA = "same memory, smaller representation"
- • DSA = "same memory, read less of it per token"

Part 6: KV-Cache — Efficient Autoregressive Generation

31

The Generation Bottleneck

- Recall: GPT-style models generate text **one token at a time** (autoregressive)
- **Naive approach**: For each new token, run the full forward pass over the **entire** sequence so far
- Step 1: Forward pass on ["The"] (1 token) → predict next
- Step 2: Forward pass on ["The", "cat"] (2 tokens) → predict next
- Step 3: Forward pass on ["The", "cat", "sat"] (3 tokens) → predict next
- At step T: process T tokens — total work to generate N tokens = $O(N^2)$ forward passes through attention
- **The waste**: At step 3, the K and V vectors for "The" and "cat" are **exactly the same** as in step 2 — we're recomputing them for nothing!
- **KV-Cache**: Cache the K and V tensors from all previous positions, only compute the new token's K and V

32

How KV-Cache Works

- **Phase 1 — Prefill**: Process the entire prompt at once (just like training) — cache all K and V
- **Phase 2 — Decode**: For each new token, compute only that token's Q, K, V
- Append the new K, V to the cache; attend new Q to **all** cached K, V
- Only one new row of Q, but it attends to the entire cached sequence
- **Savings**: Instead of recomputing T tokens' projections at step T, compute only **1 token's** projections
- The attention computation itself is still $O(T)$ per step (one Q row × T KV rows), but the projection cost drops from $O(T \cdot d^2)$ to $O(d^2)$
- The cache **grows** by one KV entry per layer per step — memory increases linearly with sequence length
- **Trade-off**: Uses more memory (storing K, V for every layer × every position) but saves enormous compute

🚀 **Interactive Demo:** kv_cache_demo.html

33

Prefill vs Decode: Two Different Bottlenecks

- LLM inference has two distinct phases with very different compute profiles:
- **Prefill phase**: Process the entire prompt in parallel → **compute-bound** (matrix multiplications)
- All prompt tokens' K, V are computed and cached in one forward pass
- Similar to training — highly parallelizable, GPU utilization is high
- **Decode phase**: Generate one token at a time → **memory-bandwidth bound** (reading KV-cache)
- Each step: compute Q for 1 token, read entire KV-cache, compute attention, generate 1 token
- The GPU spends most time **reading KV-cache from memory**, not computing — this is why generation feels slow
- **Optimization strategies**: GQA (smaller cache), KV-cache quantization, speculative decoding, PagedAttention (vLLM), MLA-style latent KV caches
- This is why **GQA + KV-cache** go hand-in-hand — GQA makes long-context generation practical

34

KV-Cache Memory Analysis

- **Per layer**: Store K and V for all cached positions
- Size = $2 \times T \times n_{kv} \times d_{\text{head}} \times \text{bytes_per_param}$
- **Example — 7B model** (32 layers, 32 Q heads, 8 KV heads, $d_{\text{head}} = 128$, float16):
- Per layer at $T = 4096$: $2 \times 4096 \times 8 \times 128 \times 2 = 16$ MB
- Total (32 layers): **512 MB** — manageable!
- Without GQA (32 KV heads): **2 GB** — 4× more!
- **At 128K context with GQA**: $32 \times 2 \times 131072 \times 8 \times 128 \times 2$ = **~16 GB**
- **At 128K context without GQA**: **~64 GB** — exceeds many GPUs!
- KV-cache management (memory allocation, eviction policies) is a major engineering challenge for serving LLMs

35

Autoregressive Generation with KV-Cache

PYTHON

```
import torch
import torch.nn.functional as F

@torch.no_grad()
def generate(model, prompt_ids, max_new_tokens, temperature=0.7, top_p=0
    """Autoregressive generation with KV-cache."""
    token_ids = prompt_ids  # (1, prompt_len)

    # Phase 1: PREFILL — process entire prompt, build initial cache
    logits, kv_cache = model.forward(token_ids, kv_cache=None)
    # kv_cache: list of (K, V) tuples, one per layer
    # Each K, V: (batch, n_kv_heads, seq_len, head_dim)

    for step in range(max_new_tokens):
        # Sample next token from last position's logits
        next_logits = logits[:, -1, :] / temperature
        probs = top_p_sample(F.softmax(next_logits, dim=-1), top_p)
        next_token = torch.multinomial(probs, num_samples=1)  # (1, 1)

        # Phase 2: DECODE — process ONLY the new token, reuse cached K,V
        logits, kv_cache = model.forward(next_token, kv_cache=kv_cache)
        # Internally: new K,V appended to cache; Q attends to full cache
        # Only 1 token processed instead of entire sequence!

        token_ids = torch.cat([token_ids, next_token], dim=1)
        if next_token.item() == tokenizer.eos_token_id:
            break

    return token_ids

# Without KV-cache: generating 100 tokens from 1000-token prompt
#   processes 1000 + 1001 + 1002 + ... + 1099 ≈ 105,000 total tokens
# With KV-cache: processes 1000 + 1 + 1 + ... + 1 = 1,100 total tokens ÷
```

Part 7: Flash Attention — Breaking the Memory Wall

37

The Memory Wall in Standard Attention

- Standard attention: $\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$
- The matrix QK^T has shape $(T \times T)$ — **quadratic** in sequence length
- For $T = 4096$: $4096^2 = 16.8\mathrm{M}$ entries per head → stored in GPU HBM (high-bandwidth memory)
- For $T = 128\mathrm{K}$: $128\mathrm{K}^2 = 16.4$ **billion** entries per head → **impossible to fit in memory**
- **GPU memory hierarchy** matters here:
- **SRAM** (on-chip): ~20–200 MB capacity, ~20 TB/s bandwidth — very fast, very small
- **HBM** (off-chip): ~40–80 GB capacity, ~2 TB/s bandwidth — large, but 10× slower than SRAM
- Standard attention writes the entire $T \times T$ matrix to slow HBM → **IO-bound, not compute-bound**
- **Analogy**: Standard = computing on paper (write results to desk, pick them up again); Flash = computing in your head (keep intermediate results in working memory)

38

Flash Attention: IO-Aware Exact Attention

- **Key insight**: We don't need to **materialize** the full $T \times T$ attention matrix in HBM
- **Flash Attention** (Dao et al., 2022): Tile the computation into small blocks that fit in SRAM
- Process Q, K, V in blocks — compute partial attention scores, accumulate results in SRAM
- Uses the **online softmax** trick: compute softmax incrementally without seeing all values at once
- Fuses all operations (matmul → softmax → matmul) into a **single GPU kernel** — no HBM round trips
- **Not an approximation**: It matches the standard attention formulation up to normal floating-point roundoff
- **Memory**: $O(T)$ instead of $O(T^2)$ — the $T \times T$ matrix **never exists** in HBM
- **Speed**: Often 2–4× faster due to far fewer HBM reads/writes (IO-bound → compute-bound)
- The causal mask is handled **implicitly** in the tiling — no explicit mask tensor needed
- This single optimization makes long-context training and inference practical

39

Flash Attention Versions and Usage (Updated for 2026)

- **FlashAttention v1** (Dao et al., 2022): Original paper, custom CUDA kernels
- **FlashAttention-2** (Dao, 2023): Better parallelism and work partitioning — common baseline in many frameworks
- **FlashAttention-3** (Shah et al., 2024): Optimized for Hopper GPUs (H100/H200), with FP8-focused kernel techniques
- **PyTorch SDPA (what most people actually use):**
- • PyTorch docs list 3 SDPA implementations: **FlashAttention-2**, **Memory-Efficient**, and **Math** fallback
- • SDPA auto-selects the best available backend based on inputs/hardware; you can override via `torch.nn.attention.sdpa_kernel`
- **Bottom line**: Use `F.scaled_dot_product_attention` everywhere — you'll get the fastest kernel available for your environment

40

Flash Attention: One Line of Code

PYTHON

```
import torch
import torch.nn.functional as F
import math

# Assume q, k, v: (batch, n_heads, seq_len, head_dim)

# === OLD WAY: Standard attention — O(T²) memory ===
def standard_attention(q, k, v, is_causal=True):
    T = q.size(-2)
    scale = math.sqrt(q.size(-1))
    attn_weights = torch.matmul(q, k.transpose(-2, -1)) / scale  # (B, H
    if is_causal:
        mask = torch.triu(torch.ones(T, T, device=q.device), diagonal=1)
        attn_weights = attn_weights.masked_fill(mask, float('-inf'))  #
    attn_weights = F.softmax(attn_weights, dim=-1)
    return torch.matmul(attn_weights, v)

# === NEW WAY: Flash / Memory-Efficient SDPA — O(T) memory when flash ke
def sdpa_attention(q, k, v):
    return F.scaled_dot_product_attention(q, k, v, is_causal=True)
    # PyTorch automatically chooses an optimized kernel on CUDA.

# NOTE: Exact bitwise equality is not guaranteed across kernels due to f
# but results should be numerically very close.
q = torch.randn(2, 8, 1024, 64, device='cuda', dtype=torch.float16)
k = torch.randn(2, 8, 1024, 64, device='cuda', dtype=torch.float16)
v = torch.randn(2, 8, 1024, 64, device='cuda', dtype=torch.float16)
out_standard = standard_attention(q, k, v)
out_sdpa = sdpa_attention(q, k, v)
print(f"Max difference: {(out_standard - out_sdpa).abs().max():.6f}")
```

Part 8: Putting It All Together

42

The Modern Transformer Block

- **Every upgrade is a drop-in replacement** — the block structure is the same, just with better components:
- **Original (2017):**
- $x \to \mathrm{MHA}(x) + x \to \mathrm{LayerNorm} \to \mathrm{ReLU\text{-}FFN}(x) + x \to \mathrm{LayerNorm}$ (post-norm)
- **Modern (2024–2026):**
- $x \to x + (\mathrm{GQA/efficient\text{-}attn})(\mathrm{RMSNorm}(x),\ \mathrm{RoPE}) \to x + \mathrm{SwiGLU}(\mathrm{RMSNorm}(x))$ (pre-norm)
- **Changes summarized:**
- • Post-norm → **Pre-norm**: Norm before sublayer, clean residual stream
- • LayerNorm → **RMSNorm**: Simpler, faster, no mean subtraction
- • Sinusoidal/Learned PE → **RoPE**: Rotation in attention, no position table
- • MHA → **GQA (sometimes)**: Fewer KV heads, smaller cache, similar quality
- • ReLU → **SwiGLU**: Gated activation, 3 weight matrices, better performance
- • Standard attention → **Flash Attention / SDPA**: IO-aware, $O(T)$ memory in flash kernel

🚀 **Interactive Demo:** modern_block_demo.html

43

The Modern Transformer Block — Complete Code

PYTHON

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ModernTransformerBlock(nn.Module):
    """A single transformer block with all modern upgrades.
    Pre-RMSNorm + GQA with RoPE + SwiGLU FFN."""
    def __init__(self, embed_dim: int, n_heads: int, n_kv_heads: int):
        super().__init__()
        self.norm1 = RMSNorm(embed_dim)
        self.attn = GroupedQueryAttention(embed_dim, n_heads, n_kv_heads
        self.norm2 = RMSNorm(embed_dim)
        self.ffn = SwiGLUFFN(embed_dim)

    def forward(self, x, freqs_cis, kv_cache=None):
        # Pre-norm: normalize BEFORE sublayer, clean residual
        h = self.norm1(x)
        attn_out, new_kv = self.attn(h, freqs_cis, kv_cache)  # GQA + Ro
        x = x + attn_out                                       # Residua

        x = x + self.ffn(self.norm2(x))                         # Pre-no
        return x, new_kv

# Compare to the 2017 original block:
# - nn.LayerNorm      → RMSNorm         ✓
# - nn.MultiheadAttn  → GQA + RoPE      ✓
# - ReLU FFN          → SwiGLU          ✓
# - Post-norm         → Pre-norm        ✓
# Same structure, better components!
```

GPT-OSS — Full Model Skeleton with Design Notes

PYTHON

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class GPTModel(nn.Module):
    """Modern GPT with all upgrades — the skeleton for GPT-OSS.

    Design choices (roughly matching the modern dense ~8B tier):
      - vocab_size  = 200,000  (byte-level BPE via tiktoken)
      - embed_dim   = 4,096
      - n_layers    = 32
      - n_heads     = 32  (Q heads)  →  head_dim = 128
      - n_kv_heads  = 8   (GQA 4:1) →  4× KV-cache savings
      - FFN hidden  ≈ 11,008  (SwiGLU, 8/3 × 4096, rounded)
      - No biases (often), no dropout (pre-training), weight tying
      - Estimated total: ~7B parameters

    Note: Many 2026 SOTA stacks add MoE + hybrid/sparse attention + MLA-
    This skeleton intentionally teaches the "classic" modern dense backb
    """
    def __init__(self, vocab_size=200_000, max_seq_len=8192,
                 embed_dim=4096, n_layers=32, n_heads=32, n_kv_heads=8):
        super().__init__()
        self.token_embed = nn.Embedding(vocab_size, embed_dim)
        # No positional embedding table — RoPE is applied inside attenti

        self.layers = nn.ModuleList([
            ModernTransformerBlock(embed_dim, n_heads, n_kv_heads)
            for _ in range(n_layers)
        ])
        self.norm = RMSNorm(embed_dim)          # Final norm (required b
        self.lm_head = nn.Linear(embed_dim, vocab_size, bias=False)
        self.lm_head.weight = self.token_embed.weight  # Weight tying →

        # Precompute RoPE frequencies once
        head_dim = embed_dim // n_heads
        self.register_buffer(
            'freqs_cis',
            precompute_freqs_cis(head_dim, max_seq_len),
            persistent=False
        )

    def forward(self, token_ids, targets=None, kv_cache=None):
```

```
        B, T = token_ids.shape
        x = self.token_embed(token_ids)        # (B, T, embed_dim)
        freqs = self.freqs_cis[:T]

        new_kv_cache = []
        for i, layer in enumerate(self.layers):
            cache_i = kv_cache[i] if kv_cache else None
            x, kv = layer(x, freqs, cache_i)
            new_kv_cache.append(kv)

        x = self.norm(x)                       # Final RMSNorm
        logits = self.lm_head(x)               # (B, T, vocab_size)

        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.view(-1, logits.size(-1)), targets.view(-1)
            )
        return logits, loss, new_kv_cache
```

Part 9: Additional Modern Techniques

46

No Biases, No Dropout (Common Pre-Training Defaults)

- **Removing biases**: The original Transformer used bias terms everywhere: $y = Wx + b$
- Many modern LLM stacks remove biases entirely: $y = Wx$ — especially in attention projections, FFN projections, and the LM head
- **Why?** Biases often provide negligible benefit at scale; RMSNorm and the residual stream already provide flexible offsets
- **Removing dropout**: The original Transformer used dropout ($p = 0.1$) after attention, after sublayers, after embedding
- Many large LLMs use **0 dropout during pre-training** — with huge datasets, memorization is less of a concern than throughput/stability
- Dropout may still be used during **fine-tuning** on smaller datasets
- Concrete examples in public configs: **Qwen3.5, GLM-5, Kimi K2.5, DeepSeek V3.2** show `attention_dropout=0.0` and `attention_bias=false`.

47

Weight Initialization and Context Extension

- **Initialization** is critical for stable training of deep transformers:
- • Standard: Xavier/Glorot or Kaiming initialization for linear layers
- • **Residual scaling**: Scale residual branch outputs by $1/\sqrt{2N}$ where N = number of layers — prevents variance growth
- • GPT-3 approach: Output projections (W_O, W_2) initialized with std $= 0.02/\sqrt{2N}$
- Bad initialization → training instability, loss spikes, or divergence — especially at large scale
- **Context length extension** — RoPE makes this much easier than learned absolute tables:
- • **Problem**: Model trained at 8K–32K context → needs to handle 128K+ at deployment
- • **NTK-aware scaling / YaRN-style scaling**: Adjust RoPE parameters + sometimes attention temperature to reduce extrapolation pain
- Concrete examples from public configs:
- • **Qwen3.5-397B-A17B**: `max_position_embeddings=262144`, `rope_theta=10000000`
- • **Kimi K2.5**: `max_position_embeddings=262144`, `rope_theta=50000`, YaRN scaling `factor=64`
- • **GLM-5**: `max_position_embeddings=202752`, `rope_theta=1000000`
- • **MiniMax M2.5**: `max_position_embeddings=196608`, `rope_theta=5000000`, `rotary_dim=64`
- • **DeepSeek V3.2**: `max_position_embeddings=163840`, `rope_theta=10000`, YaRN scaling `factor=40`
- For GPT-OSS: Train at a moderate context length, extend later with these techniques

2026 Add-ons: MoE + MLA + Sparse/Hybrid Attention (Beyond the "Classic" Block)

- Your 6 upgrades are the **core modern backbone** — but many 2026 SOTA models add extra machinery on top:
- **1) Sparse MoE (Mixture of Experts):**
- • Replaces the dense FFN with a router + experts → much higher total parameter count with similar per-token FLOPs
- • Examples (from configs/model cards): **Qwen3.5** (`num_experts=512`, `num_experts_per_tok=10`), **MiniMax M2.5** (`num_local_experts=256`, `num_experts_per_tok=8`), **Kimi K2.5** (`n_routed_experts=384`, `num_experts_per_tok=8`, plus `n_shared_experts=1`), **GLM-5** (`n_routed_experts=256`, `num_experts_per_tok=8`, plus `n_shared_experts=1`), **DeepSeek V3.2** (`n_routed_experts=256`, `num_experts_per_tok=8`, plus `n_shared_experts=1`).
- **2) MLA (Multi-head Latent Attention) & KV compression ideas:**
- • Goal: reduce KV-cache cost while keeping strong long-context quality
- • DeepSeek V3.2 tech report explicitly uses **MLA** and illustrates both MHA/MQA modes
- • Kimi K2.5 model card lists attention mechanism as **MLA**
- **3) Sparse / Hybrid Attention:**
- • Instead of attending to all previous tokens, select a subset (top-k) or mix linear attention with full attention
- • DeepSeek V3.2 tech report: **DSA** = Lightning Indexer + Top-k selector (2048 selected KV tokens per query token in the report)
- • Qwen3.5 config shows a hybrid layer schedule mixing **linear_attention** and **full_attention** blocks
- **Takeaway:** Your block is still the foundation. These add-ons mainly change *routing* (MoE) and *attention efficiency* (MLA/sparse/hybrid).

Part 10: Common Pitfalls

50

Mistakes You'll Make (and How to Avoid Them)

- **1. Forgetting the final RMSNorm** — Pre-norm means the output of the last layer is *un-normalized*; you must add `self.norm(x)` before the LM head or logits will explode
- **2. Applying RoPE to V** — RoPE goes on Q and K only; applying it to V adds position-dependent distortion to the aggregated content
- **3. Wrong SwiGLU hidden dim** — Using $4d$ instead of $\frac{8}{3}d$ gives ~50% more FFN parameters than intended; always round to a hardware-friendly multiple (e.g., 256)
- **4. Not rounding hidden dim** — Unaligned dimensions (e.g., 10923) cause poor GPU utilization; round to 128 or 256
- **5. Broadcasting RoPE frequencies wrong** — `freqs_cis` must align with `(batch, seq_len, n_heads, head_dim/2)`; a wrong unsqueeze silently broadcasts to the batch or head dim
- **6. KV-cache position offset** — During decode, the new token is at position T_{cached}, not position 0; use `freqs_cis[start_pos:start_pos+1]`
- **7. Mixing up pre-norm and post-norm** — If you normalize after the residual add, you've built a post-norm block; the gradient highway disappears
- **8. Weight tying shape mismatch** — `lm_head` and `token_embed` must have the same `vocab_size × embed_dim`; mismatched vocab sizes cause silent shape errors

Part 11: Real Model Configurations

52

State-of-the-Art Architectures Compared (2026, Publicly Documented)

- At the frontier, the **core primitives** you learned still show up everywhere: pre-norm + RoPE + gated MLPs + KV-cache optimizations + fused attention kernels.
- What varies is **(a) MoE routing**, and **(b) attention efficiency tricks** (GQA vs MLA vs sparse/hybrid attention).
-
- **Examples with public configs + official docs:**
- • **Qwen3.5-397B-A17B (text config)**: `num_hidden_layers=60`, `hidden_size=4096`, `num_attention_heads=32`, `num_key_value_heads=2`, `max_position_embeddings=262144`, MoE `num_experts=512`, `num_experts_per_tok=10`, RoPE `rope_theta=10000000`, and a hybrid schedule mixing `linear_attention` + `full_attention` layers
- • **GLM-5**: `num_hidden_layers=78`, `hidden_size=6144`, `num_attention_heads=64`, `num_key_value_heads=64`, `max_position_embeddings=202752`, MoE `n_routed_experts=256`, `num_experts_per_tok=8`, RoPE `rope_theta=1000000`, and integrates DeepSeek Sparse Attention (DSA) per model card
- • **Kimi K2.5**: model card summarizes 1T total / 32B active, 61 layers, hidden size 7168, 64 heads, 384 experts (8 selected + 1 shared), context length 256K; HF config shows YaRN `factor=64` and `rope_theta=50000`
- • **MiniMax M2.5**: `num_hidden_layers=62`, `hidden_size=3072`, `num_attention_heads=48`, `num_key_value_heads=8`, `max_position_embeddings=196608`, MoE `num_local_experts=256`, `num_experts_per_tok=8`, RoPE `rope_theta=5000000`, `rotary_dim=64`
- • **DeepSeek V3.2**: HF config includes `num_hidden_layers=61`, `hidden_size=7168`, `num_attention_heads=128`, `max_position_embeddings=163840`, MoE `n_routed_experts=256`, `num_experts_per_tok=8`, RoPE `rope_theta=10000` with YaRN scaling `factor=40`; tech report introduces **DSA** with a **Lightning Indexer** and states DSA is instantiated under **MLA**
-
- Across these, the block wiring is stable — the big innovations are in **MoE routing** and **attention variants** (hybrid linear, sparse top-k selection, MLA/KV compression).

Parameter Count Breakdown

- **Where do the parameters live?** Breakdown for a typical dense ~7B decoder-only model:
- • **Token embeddings**: 32K × 4096 = **131M** (1.9%)
- • **Attention (Q, K, V, O)**: 32 layers × $(4096^2 + 3 \times 4096 \times 1024)$ ≈ **2.1B** (31%) — note K, V smaller with GQA
- • **FFN (SwiGLU W_1, W_2, W_3)**: 32 layers × 3 × 4096 × 11008 ≈ **4.3B** (63%)
- • **RMSNorm**: 32 layers × 2 × 4096 + 4096 = **0.26M** (0.004%)
- • **LM Head**: weight-tied with embeddings = **0** extra params
- **Key insight**: The FFN dominates — it's ~2/3 of all parameters
- **FLOPs per token** ≈ $2\times$ parameter count (rough rule of thumb for forward pass)
- For 7B params: ~14 GFLOPs per token; full training ≈ $6 \times 2 \times 7\text{B} \times 2\text{T} \approx 1.7 \times 10^{23}$ FLOPs

54

Summary

55

What You Now Understand

- **Pre-Norm**: Normalize before sublayers — clean residual gradient highway — stable deep training
- **RMSNorm**: Drop mean subtraction and bias from LayerNorm — ~10–15% faster, similar quality in many LLM stacks
- **SwiGLU**: Gated FFN with Swish activation — 3 weight matrices, hidden dim $= \frac{8}{3}d$ — consistently better than ReLU/GELU
- **RoPE**: Encode position via rotation of Q and K — relative position from dot products — no learned position parameters
- **GQA**: Share KV heads across groups of Q heads — n_h/n_{kv}× KV-cache reduction — negligible quality loss
- **KV-Cache**: Cache K and V from previous steps — process only new token during decode — ~100× less redundant work
- **Prefill vs Decode**: Two distinct inference phases — compute-bound vs memory-bandwidth bound
- **Flash Attention / SDPA kernels**: Tile attention into SRAM blocks — same math up to roundoff — much less IO — $O(T)$ memory in flash kernel
- **Modern defaults**: Often no biases, often no dropout (pre-training), residual scaling initialization
- **Key insight**: Many upgrades are **drop-in replacements** — the decoder-only, autoregressive, next-token-prediction architecture is unchanged
- You can now read the public configs/tech reports of **Qwen3.5, GLM-5, Kimi K2.5, MiniMax M2.5, DeepSeek V3.2** and understand the core components

The Road to GPT-OSS

- ✅ **Part 1**: Original Transformer architecture (2017) — attention, FFN, residuals, norms
- ✅ **Part 2**: NLP fundamentals — tokenization, embeddings, loss, metrics, output pipeline
- ✅ **Part 3**: Modern transformer upgrades — RoPE, RMSNorm, SwiGLU, GQA, KV-cache, Flash Attention
- 📍 **Part 4**: GPT-OSS from scratch — data pipeline, tokenizer training, model implementation

57

Interactive Demos

- 📐 **Activations & Normalization:**

  🚀 **Interactive Demo:** activation_functions_demo.html

- 🔄 **Rotary Position Embeddings:**

  🚀 **Interactive Demo:** rope_demo.html

- 🧠 **Attention Head Grouping:**

  🚀 **Interactive Demo:** gqa_demo.html

- ⚡ **KV-Cache Visualization:**

  🚀 **Interactive Demo:** kv_cache_demo.html

- 🏗️ **Modern Transformer Block:**

  🚀 **Interactive Demo:** modern_block_demo.html

58

Supplementary Resources

🚀 **Interactive Demo:** handout.html

59